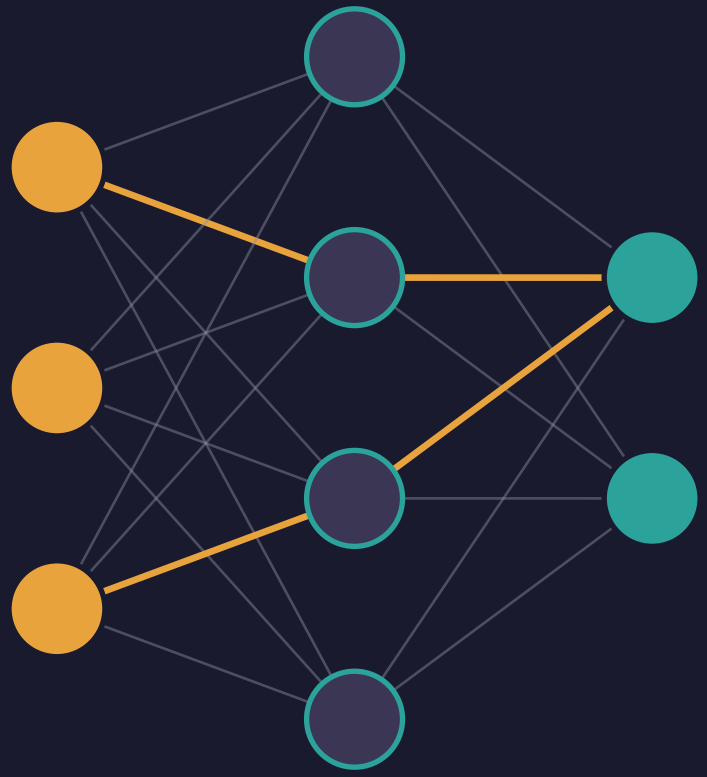
LECTURE 1 · INTRODUCTORY SESSION

Deep Learning & Reinforcement Learning

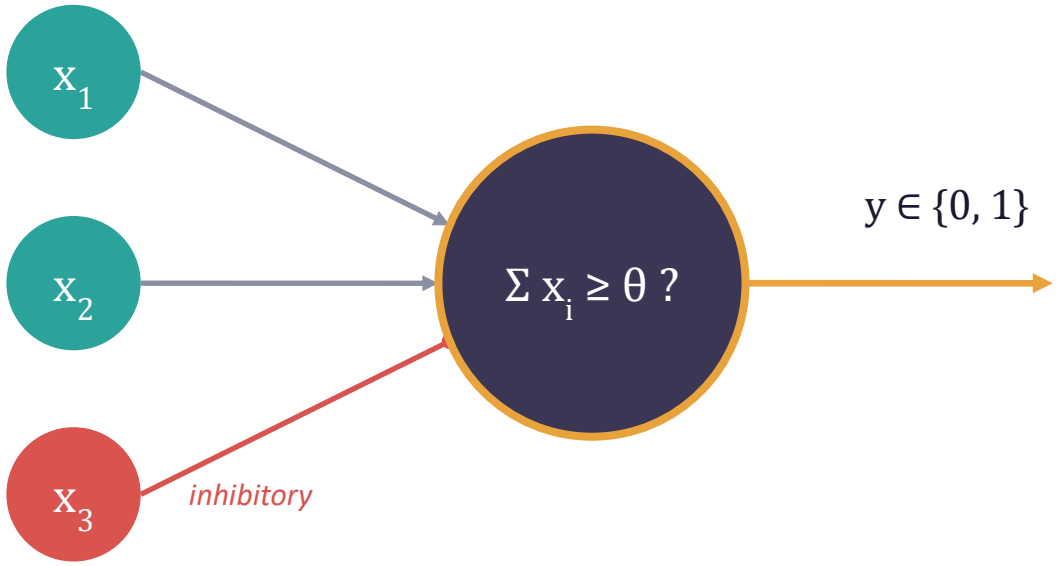
From artificial neurons to agentic AI — a guided tour of 80 years of ideas



The first artificial neuron: McCulloch–Pitts

1943

- **McCulloch & Pitts (1943)**, "A Logical Calculus of the Ideas Immanent in Nervous Activity" — the first mathematical model of a neuron
- Binary inputs $x_i \in \{0, 1\}$, each either excitatory or inhibitory
- The unit fires ($y = 1$) when the sum of active inputs reaches a threshold θ; an active inhibitory input blocks firing
- Networks of such units can compute any Boolean function — the brain as a logic machine
- **No learning:** thresholds and connections are wired by hand
- **Hebb (1949):** connections between neurons that fire together are strengthened — the first learning principle



Logic gates from a single neuron

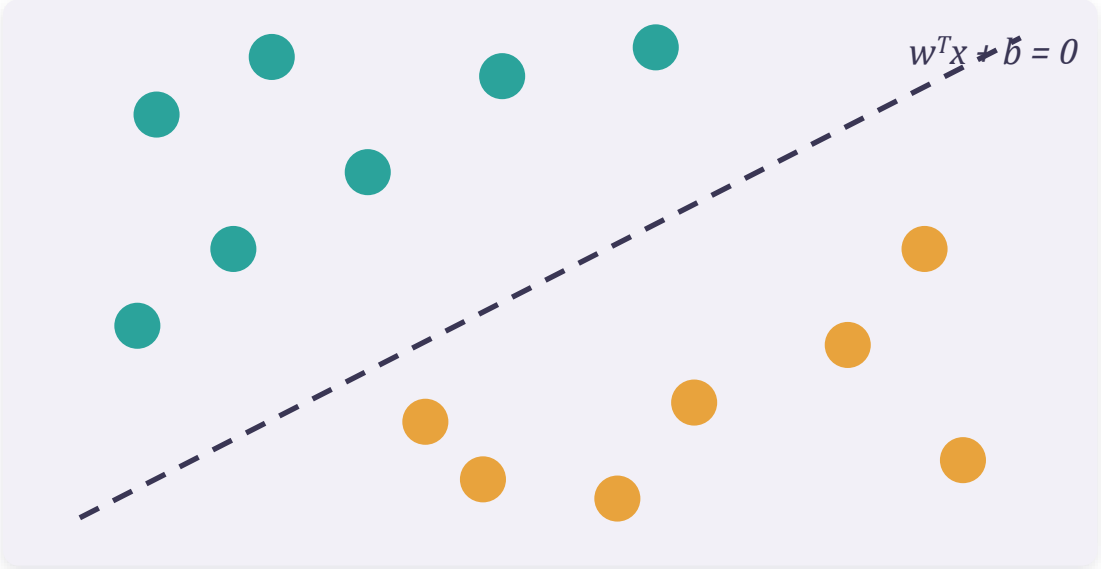
AND	OR	NOT
$\theta = 2$	$\theta = 1$	inhibitory input

Rosenblatt's perceptron: a neuron that learns

1957–58

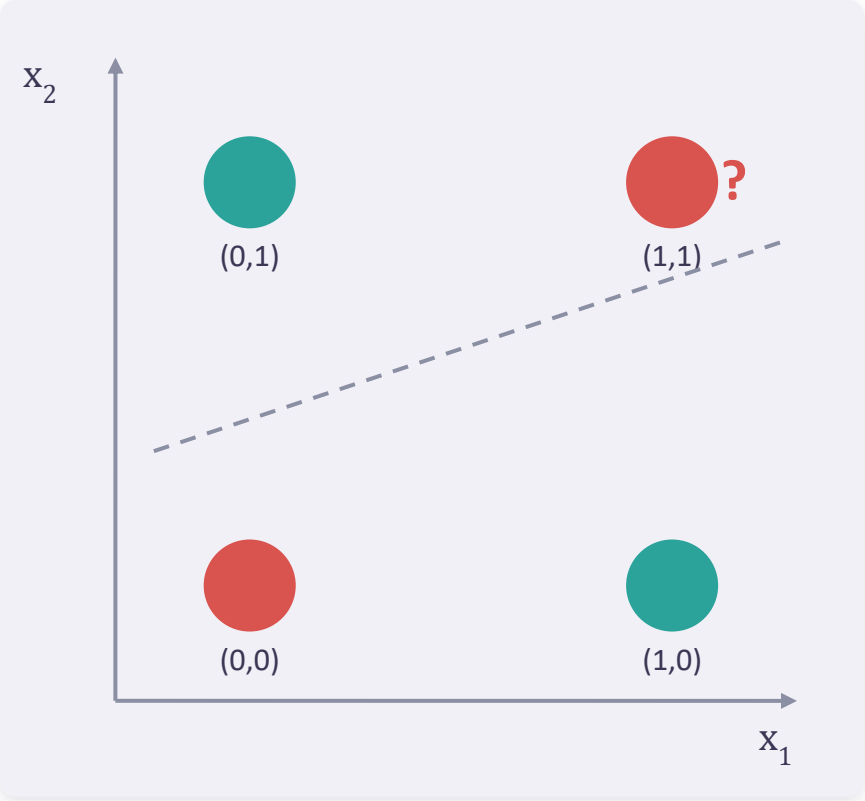
- **Frank Rosenblatt** (Cornell Aeronautical Laboratory) adds real-valued, **learnable** weights and a bias
- A linear classifier: predict $\hat{y} = \text{sign}(w^T x + b)$
- **Error-driven learning:** on each mistake, move the weights toward the correct answer
- **Convergence theorem** (Novikoff, 1962): if the classes are linearly separable, training stops after finitely many mistakes
- **Mark I Perceptron** (1960): custom hardware with a 20×20 photocell "retina" and motor-driven potentiometers as weights
- Enormous hype — 1958 press coverage promised machines that would soon walk, talk and see

$$\hat{y} = \text{sign}(w^T x + b)$$
$$w \leftarrow w + \eta\,(y - \hat{y})\,x$$
$$b \leftarrow b + \eta\,(y - \hat{y})$$



“Perceptrons” and the first AI winter

1969–80



XOR: no single line separates ● from ●

- **Minsky & Papert, “Perceptrons” (1969):** a rigorous analysis of what single-layer perceptrons can and cannot compute
- They cannot represent XOR, parity or connectedness — anything not linearly separable
- Multi-layer networks could, but nobody knew how to train them
- Widely read as “neural networks are a dead end” → funding shifted to symbolic AI

1969
“Perceptrons” is published

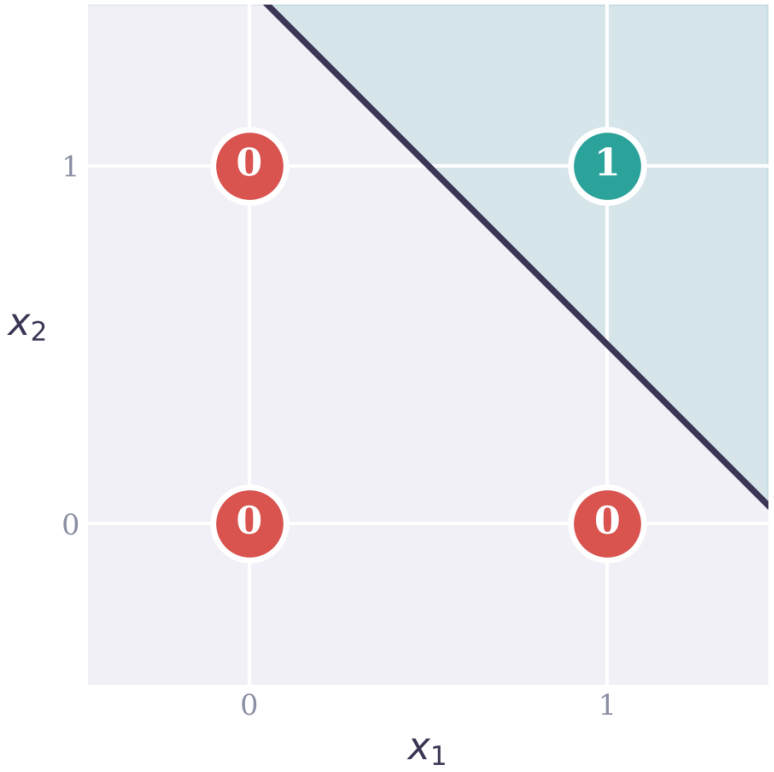
1973
Lighthill Report criticises AI in the UK

1974
DARPA cuts open-ended AI funding

First AI winter
≈ 1974–1980

“Perceptrons” and Boolean Logic

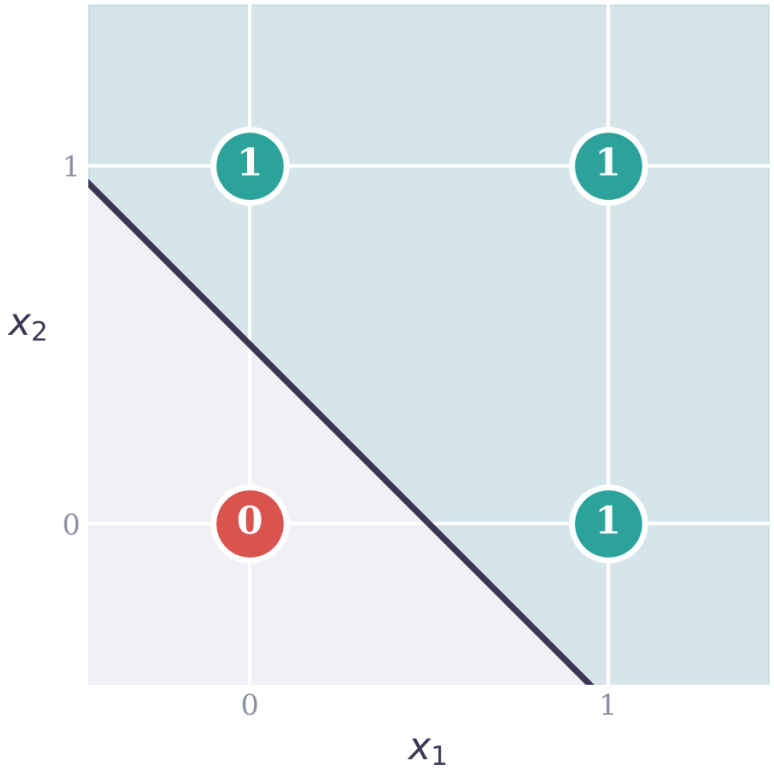
AND



w = (1, 1), b = −1.5

Linearly separable

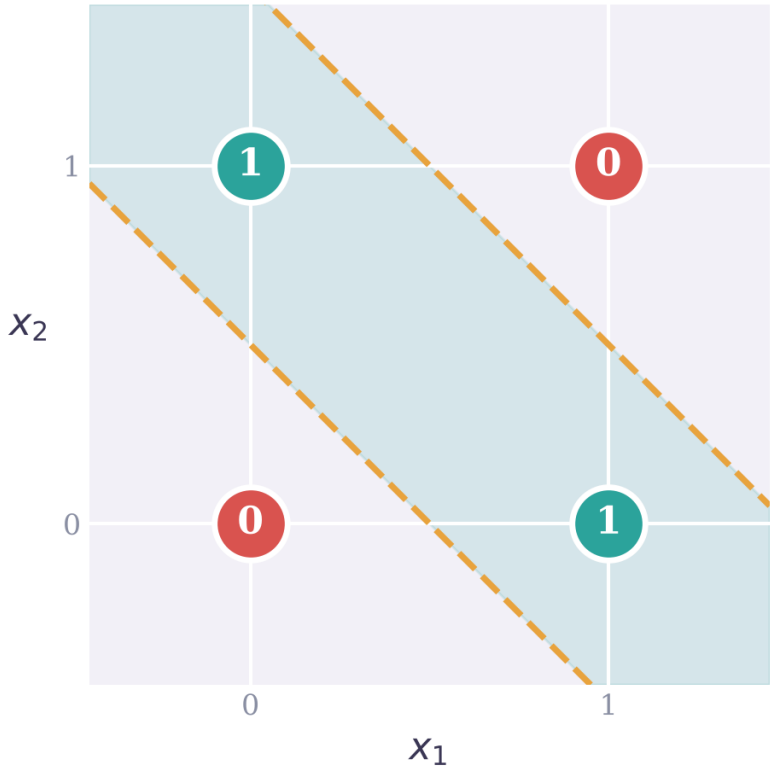
OR



w = (1, 1), b = −0.5

Linearly separable

XOR



two lines needed → hidden layer

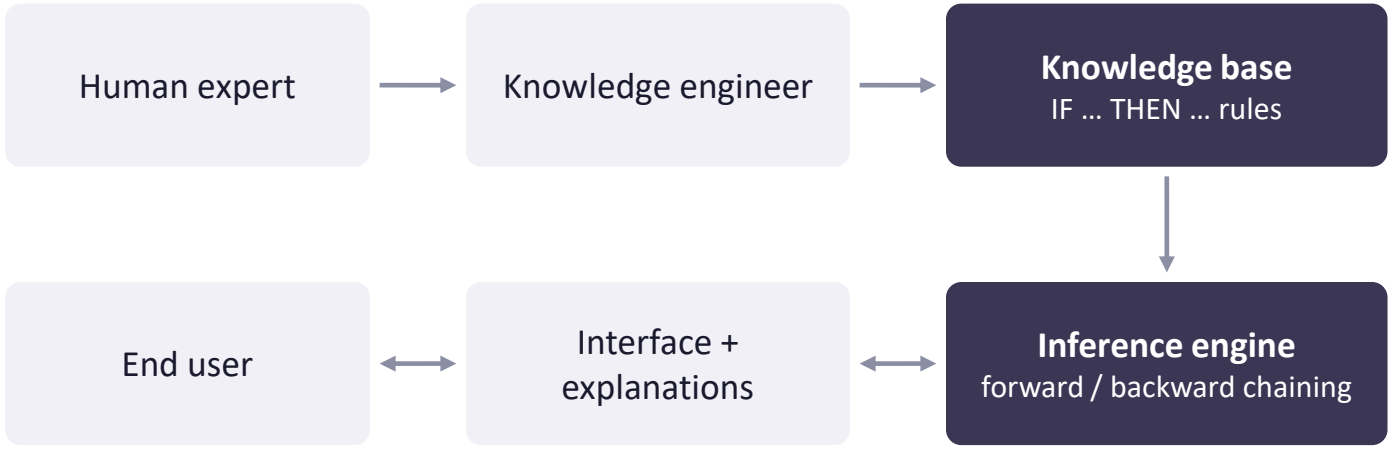
Not linearly separable

Perceptron: output 1 if $w_1x_1 + w_2x_2 + b > 0$

output 1 output 0

Symbolic AI takes over: expert systems

1970s–80s



DENDRAL

1965

Inferred molecular structure from mass-spectrometry data

MYCIN

1970s

≈600 rules to diagnose blood infections and suggest antibiotics

XCON / R1

1980

Configured DEC VAX computer orders; big reported savings

Why it collapsed

- **Knowledge-acquisition bottleneck:** rules had to be extracted from experts by hand
- Brittle outside a narrow domain; no common sense
- Could not learn from data or handle uncertainty well
- 1987: Lisp-machine market collapses → **second AI winter** (≈1987–93)

Meanwhile… backpropagation is popularised (Rumelhart, Hinton & Williams, 1986), quietly reviving multi-layer networks

The statistical learning era

1990s–2000s

SVMs & kernels

Cortes & Vapnik (1995): max-margin classifiers; the kernel trick gives non-linear boundaries via convex optimisation

Ensembles

AdaBoost (1995), Random Forests (2001): many weak learners combined into a strong one

Why it won the 1990s

- Convex problems → global optimum, reproducible
- Worked with small datasets and CPUs
- Hand-crafted features (SIFT, HOG, MFCCs) did the heavy lifting

Graphical models

Bayesian networks (Pearl, 1988), HMMs: reasoning under uncertainty; HMMs powered speech recognition

Learning theory

VC dimension and PAC learning: bounds that explain why a model generalises. Important ideas in Statistical Learning

Neural nets, meanwhile

- Deep nets were hard to train: vanishing gradients (Hochreiter, 1991)
- Bright spots: LeNet-5 read bank cheques (1998); LSTM (1997)

The return of neural networks: deep learning

2006–2012

2006

Hinton et al.: deep belief nets — layer-wise pre-training makes deep networks trainable

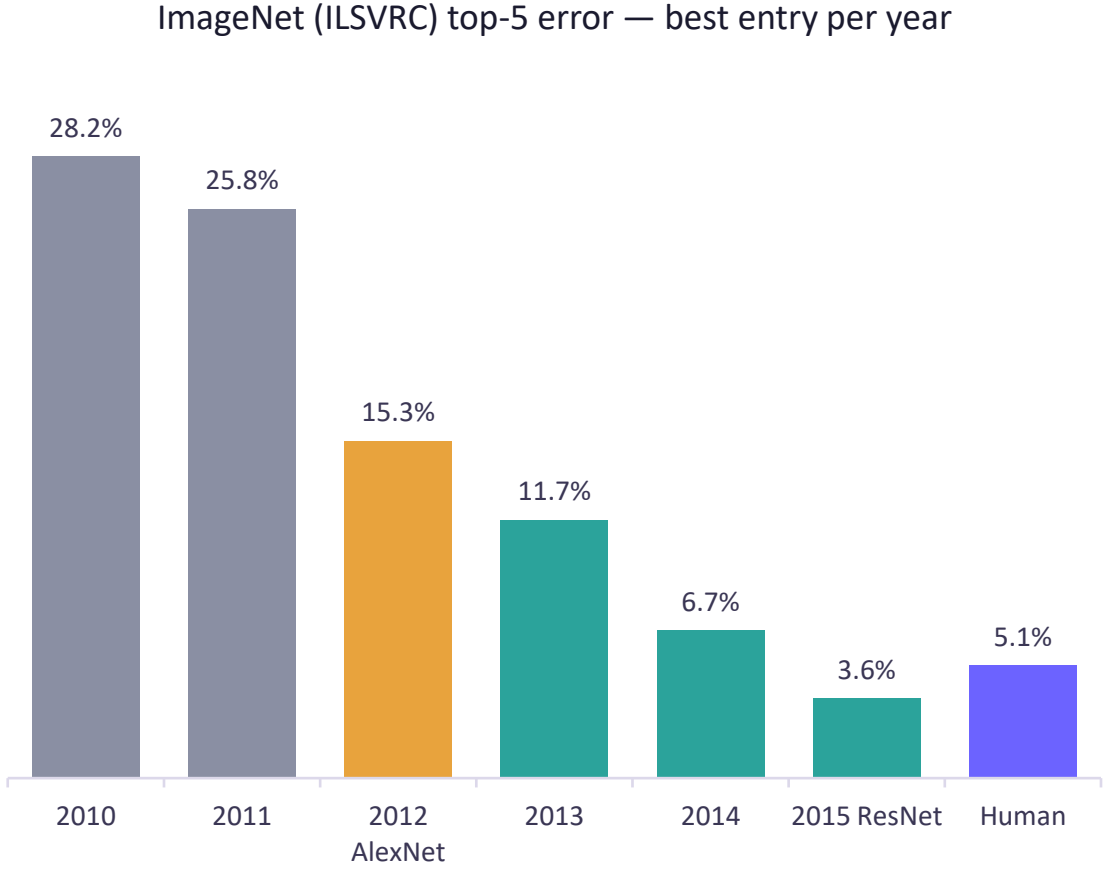
15.3%

AlexNet (2012) top-5 error on ImageNet, vs 26.2% for the runner-up — the field switched almost overnight

Data
ImageNet (2009): millions of labelled images

Compute
GPUs: AlexNet trained on two gaming cards

Algorithms
ReLU, dropout, better initialisation



Recognition: 2018 Turing Award (Bengio, Hinton, LeCun) · 2024 Nobel Prize in Physics (Hopfield, Hinton)

Three ways to learn from data

Core ideas

Supervised

$\{(x_i, y_i)\}$ *inputs + labels*

Learn a mapping f: x → y from labelled examples that generalises to unseen inputs.

OBJECTIVE

$$\min_\theta \Sigma_i \, \ell(f_\theta(x_i), y_i)$$

Image classification, spam filtering, price regression

Unsupervised

$\{x_i\}$ *inputs only*

Discover structure without labels: clusters, low-dimensional manifolds, the data density p(x).

OBJECTIVE

$$\max_\theta \Sigma_i \log p_\theta(x_i)$$

k-means, PCA, Gaussian mixtures, anomaly detection

Self-supervised

$\{x_i\}$ *labels made from the data*

Hide part of the input and predict it. A pretext task turns raw data into supervision at scale.

OBJECTIVE

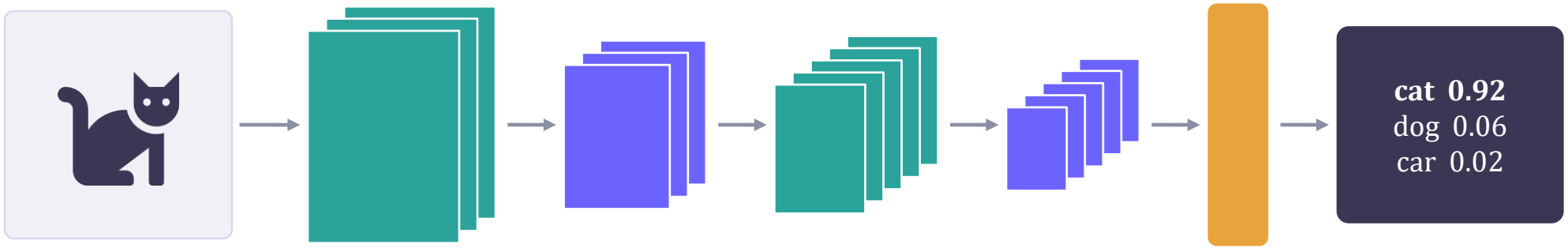
$$\max_\theta \Sigma_t \log p_\theta(x_t \mid x_{<t})$$

Next-token prediction (GPT), masked tokens (BERT), contrastive learning (SimCLR)

+ Reinforcement learning: no fixed dataset at all — an agent learns from rewards by interacting with an environment

Convolutional neural networks

1980–2015



Input image → Conv + ReLU → Pool → Conv + ReLU → Pool → Fully connected → Softmax output

Local receptive fields

Each unit sees a small patch:

$$(I * K)(i, j) = \Sigma_{m,n}\, I(i+m, j+n)\, K(m, n)$$

Weight sharing

The same filter slides over the whole image → far fewer parameters and translation equivariance

Hierarchy + pooling

Edges → textures → parts → objects; pooling adds robustness to small shifts

1980 Neocognitron (Fukushima)

1998 LeNet-5 (LeCun)

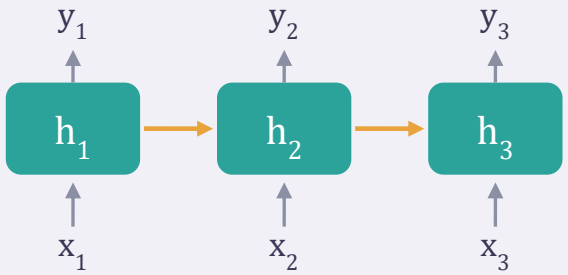
2012 AlexNet

2014 VGG, GoogLeNet

2015 ResNet: skip connections, 152 layers

Beyond CNNs: sequences, compression, generation

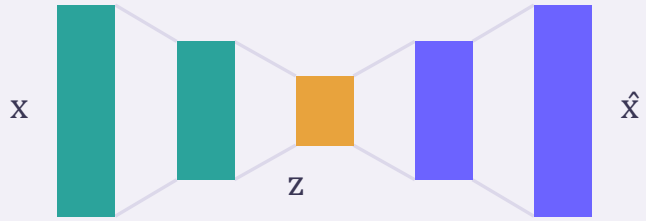
1990–2014



Recurrent nets (RNN / LSTM)

$h_t = \tanh(W\,h_{t-1} + U\,x_t + b)$

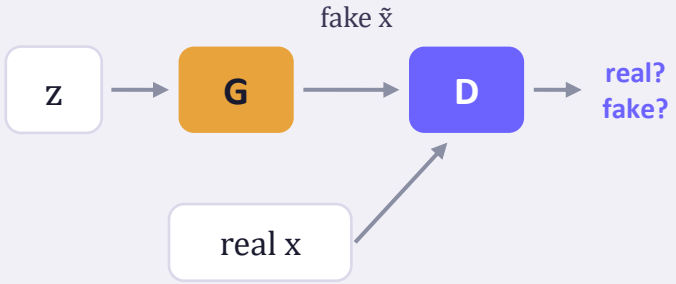
- Same weights at every time step → any sequence length
- Vanishing gradients → **LSTM** (1997), **GRU** (2014)
- Powered speech and translation (seq2seq, 2014), but sequential and slow to train



Autoencoders

$z = f_\varphi(x),\ \hat{x} = g_\theta(z),\ \min \|x - \hat{x}\|^2$

- A bottleneck forces a compact representation
- **VAE** (2013): a probabilistic latent space makes it generative
- Uses: compression, denoising, anomaly detection, latent diffusion



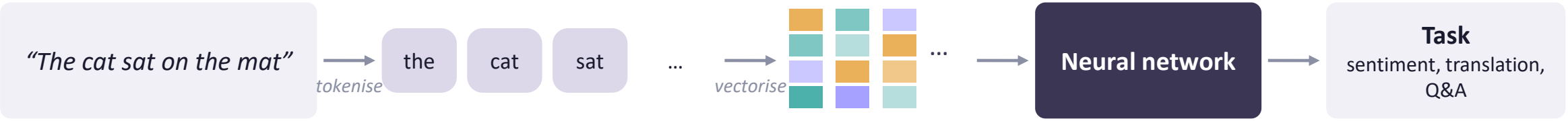
Generative adversarial nets

$\min_G \max_D E[\log D(x)] + E[\log(1 - D(G(z)))]$

- Goodfellow et al. (2014): generator vs. discriminator in a two-player game
- Photorealistic images (e.g. StyleGAN faces)
- Unstable training, mode collapse; diffusion models took over image generation (≈2021)

Natural language processing: words as vectors

1950s–2017



1 Counting words

cat = [0, 0, 1, 0, ..., 0]
dog = [0, 1, 0, 0, ..., 0]
cat · dog = 0 (no similarity)

One-hot vectors, bag-of-words and TF-IDF: sparse vectors as long as the vocabulary. Word order and meaning are lost.

2 Word embeddings



word2vec (2013), GloVe (2014): dense ≈300-d vectors learned by predicting nearby words. Similar words end up close: king − man + woman ≈ queen.

3 Contextual vectors

river **bank** → [0.8, −0.1, ...]

bank loan → [−0.4, 0.9, ...]

A static vector gives "bank" one meaning. Contextual models compute each word's vector from its sentence: RNNs + attention (2014), then Transformers (2017).

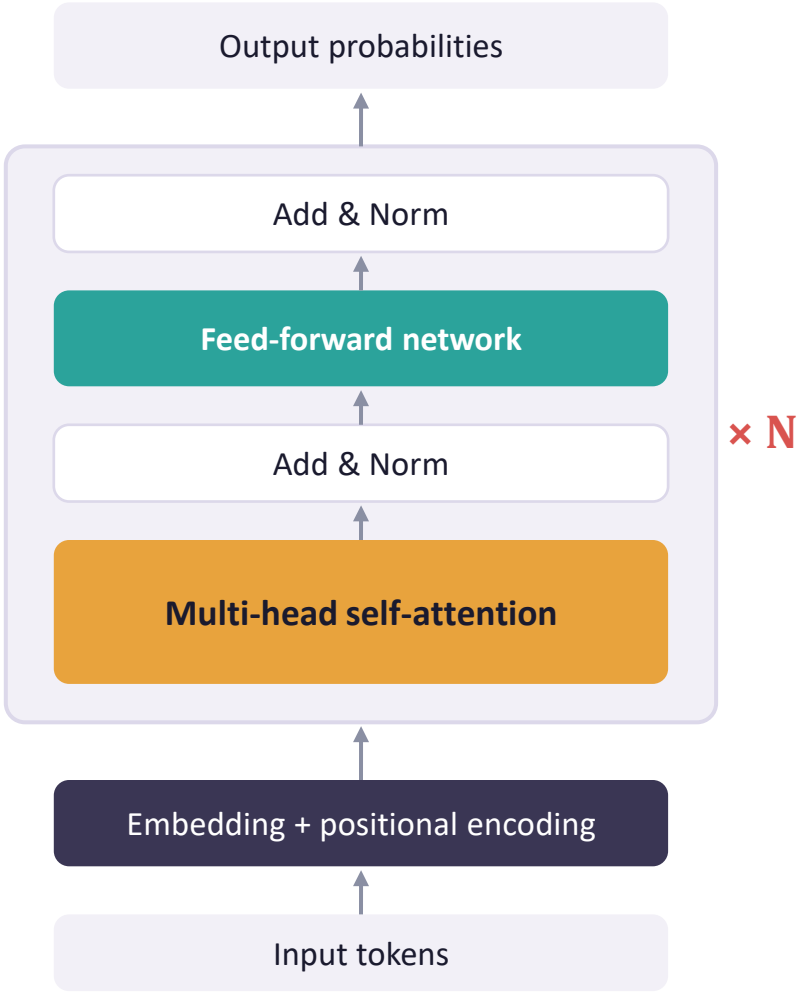
Next — the Transformer: build contextual vectors by letting every word attend directly to every other word in the sentence.

Transformers: attention is all you need

2017–

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k})\, V$$

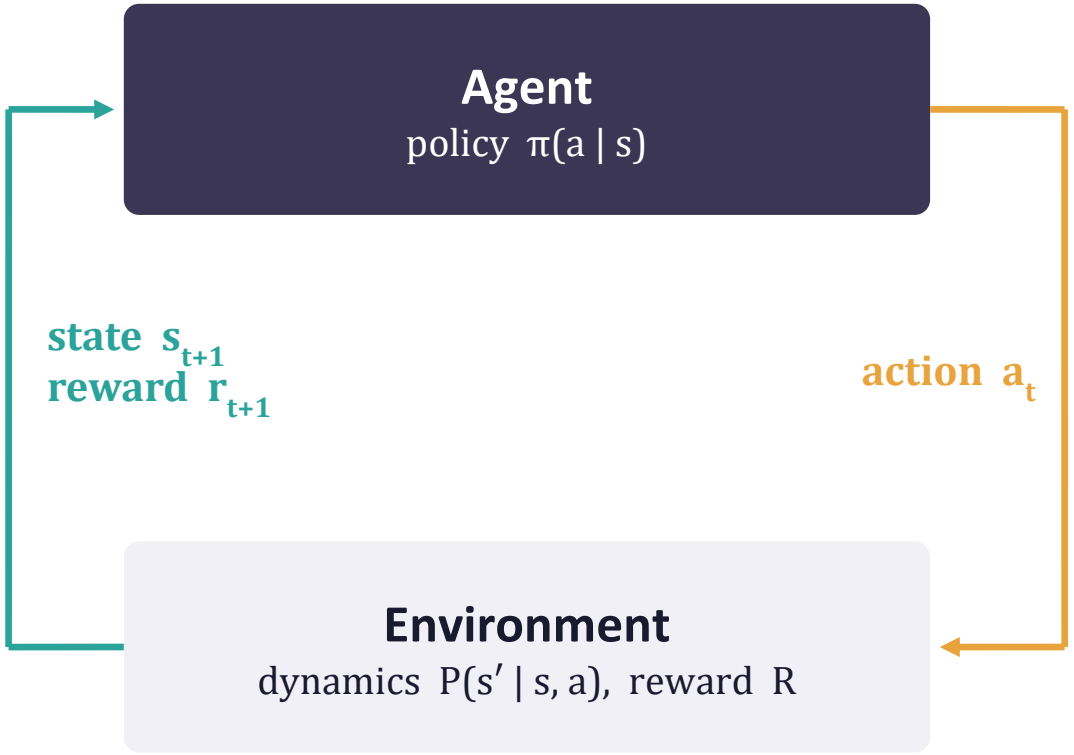
- **Vaswani et al. (2017):** drop recurrence entirely; every token attends directly to every other token
- Multi-head attention learns several kinds of relations in parallel; positional encodings supply word order
- Fully parallel training on GPUs → scales to billions of parameters and trillions of tokens
- **Scaling laws** (Kaplan et al., 2020): loss falls predictably with more data, parameters and compute
- Cost: attention is $O(n^2)$ in sequence length



Encoder-only	Decoder-only	Encoder–decoder	Vision
BERT (2018)	GPT family	T5 (2019)	ViT (2020)

Reinforcement learning: learning by interaction

Part 2



Markov decision process (S, A, P, R, γ)

Policy: $\pi(a \mid s)$

Return: $G_t = \Sigma_{k \geq 0} \gamma^k r_{t+k+1}$

Goal: $\pi^* = \text{argmax}_\pi E_\pi[G_0]$

How RL differs from supervised learning

- No labels — only a scalar reward, often delayed (**credit assignment**)
- **Exploration vs. exploitation:** try new actions or use what works?
- The data depend on the agent's own actions

The Bellman equations

1957

1 · Bellman expectation equation — evaluate a policy

$$V^{\pi}(s) = \Sigma_a \, \pi(a|s) \, \Sigma_{s'} \, P(s'|s,a) \, [\, R(s,a,s') + \gamma \, V^{\pi}(s') \,]$$

2 · Bellman optimality equation — the best possible values

$$Q^*(s,a) = E_{s'} \, [\, r + \gamma \, \max_{a'} \, Q^*(s',a') \,]$$

3 · Q-learning (Watkins, 1989) — learn Q* from samples

$$Q(s,a) \leftarrow Q(s,a) + \alpha \, [\, r + \gamma \, \max_{a'} \, Q(s',a') - Q(s,a) \,]$$

The key idea

Value now = reward now + γ × value of where you land next

- **Model known** → dynamic programming (value / policy iteration)
- **Model unknown** → sample transitions: TD learning, Q-learning
- **Q as a deep net** → deep RL (DQN)

1957 Bellman: dynamic programming

1988 TD learning (Sutton)

1989 Q-learning

1992 TD-Gammon

2015 DQN masters Atari

2016 AlphaGo beats Lee Sedol

2022 RLHF aligns LLMs

Reinforcement learning in the real world

Applications



Self-driving cars

- Decision-making: merging, lane changes, unprotected turns
- Policies trained and stress-tested in simulation
- RL fine-tuning on top of imitation-learned driving

Reality check
Production stacks rely mostly on supervised perception and imitation; trial-and-error on real roads is unsafe.



Robotics

- Legged locomotion learned in simulation (quadrupeds, humanoids)
- Dexterous manipulation: a robot hand solving a Rubik's Cube (OpenAI, 2019)
- Sim-to-real transfer via domain randomisation

Reality check
Sample-hungry and sensitive to the sim-to-real gap; now combined with imitation and foundation models.



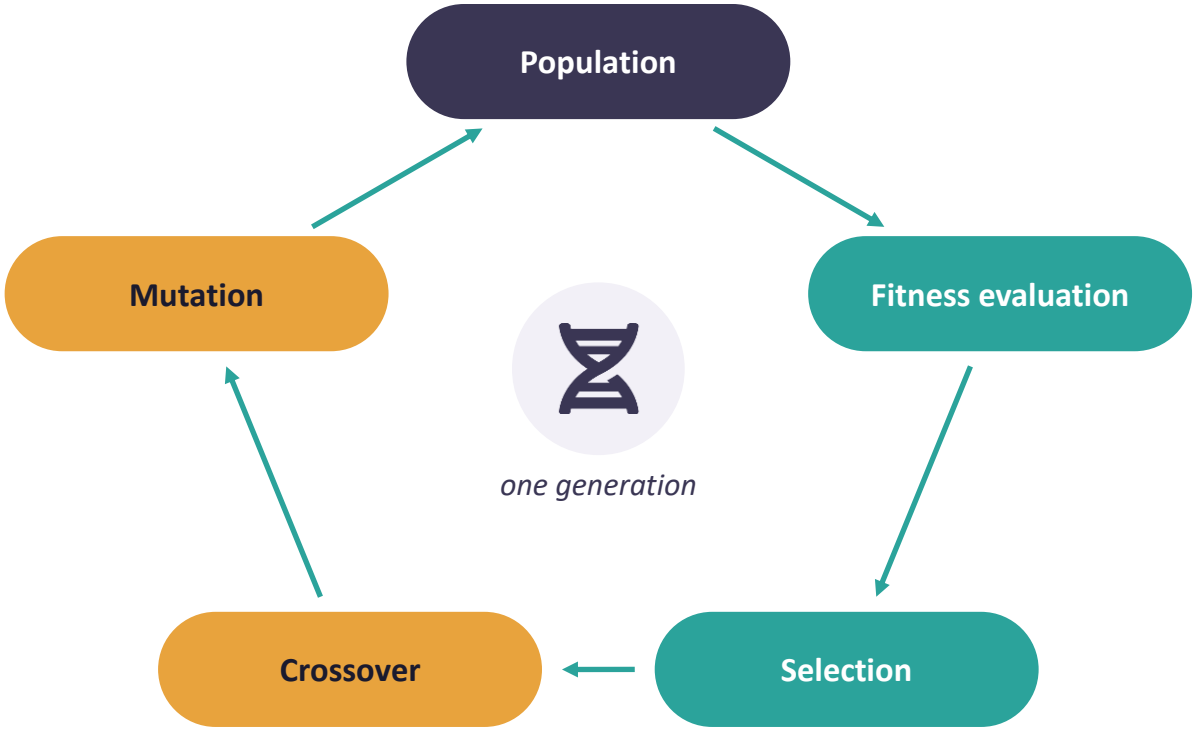
Economics & business

- Dynamic pricing, ad bidding, recommendations for long-term value
- Inventory, supply chains, portfolio management
- DeepMind cut data-centre cooling energy by up to 40% (2016)
- Policy design: the "AI Economist" (Salesforce, 2020)

Reality check
Markets shift and exploration is costly, so bandits and offline RL dominate in practice.

Bonus: genetic algorithms

1975



Where they shine

- Origins: John Holland, “Adaptation in Natural and Artificial Systems” (1975)
- NASA’s evolved ST5 antenna flew in space (2006)
- Scheduling, routing, design; neuroevolution (NEAT, 2002)

Why gradient-based deep learning won

- No gradients: black-box search scales poorly to millions of parameters
- Every fitness evaluation is costly → sample-hungry
- Backprop + GPUs made exact gradients cheap

Comeback: evolution strategies as an RL alternative (OpenAI, 2017) and LLM-guided evolutionary search (DeepMind’s AlphaEvolve, 2025)

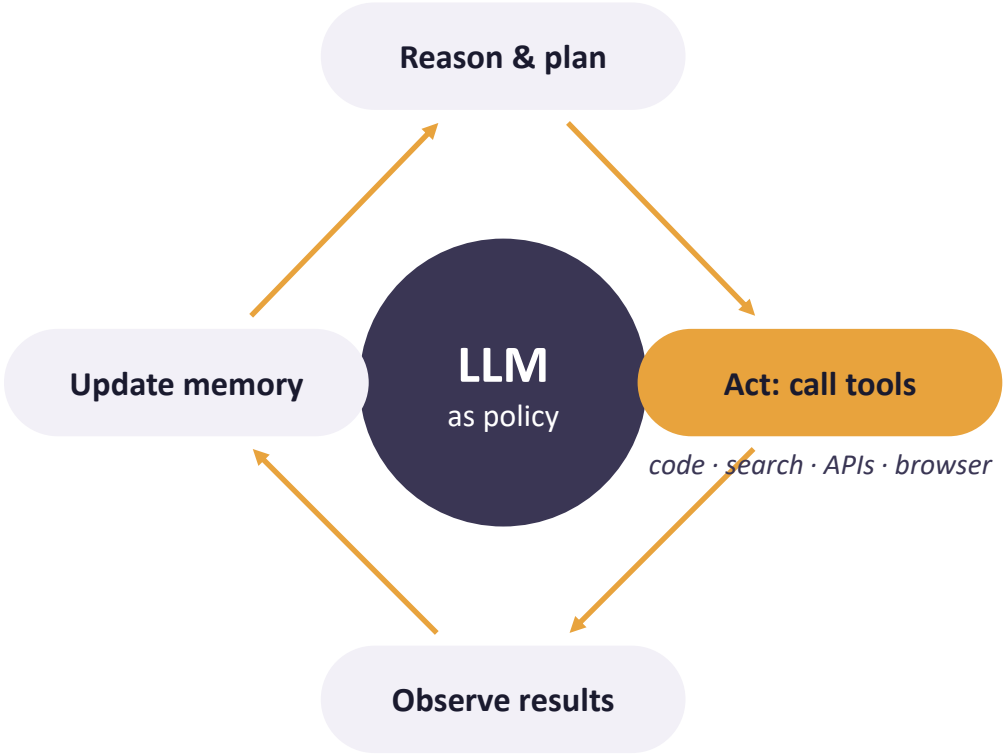
Today: foundation models and agentic AI

2020s

1 **Pre-training**
Transformer + self-supervised next-token prediction on web-scale text, code and images

2 **Post-training**
Instruction tuning, RLHF, and RL with verifiable rewards (maths, code)

3 **Reasoning**
Models trained to think step by step and spend more compute at test time



An agent = an LLM policy in an RL-style loop

The whole lecture converges here: transformers (architecture) + self-supervision (pre-training) + reinforcement learning (post-training and agency). Open problems: reliability, evaluation, safety and alignment, cost.